

Function-Oriented Software Design

Solved and Practice Exercises

Formulae Calculator (FC) Software

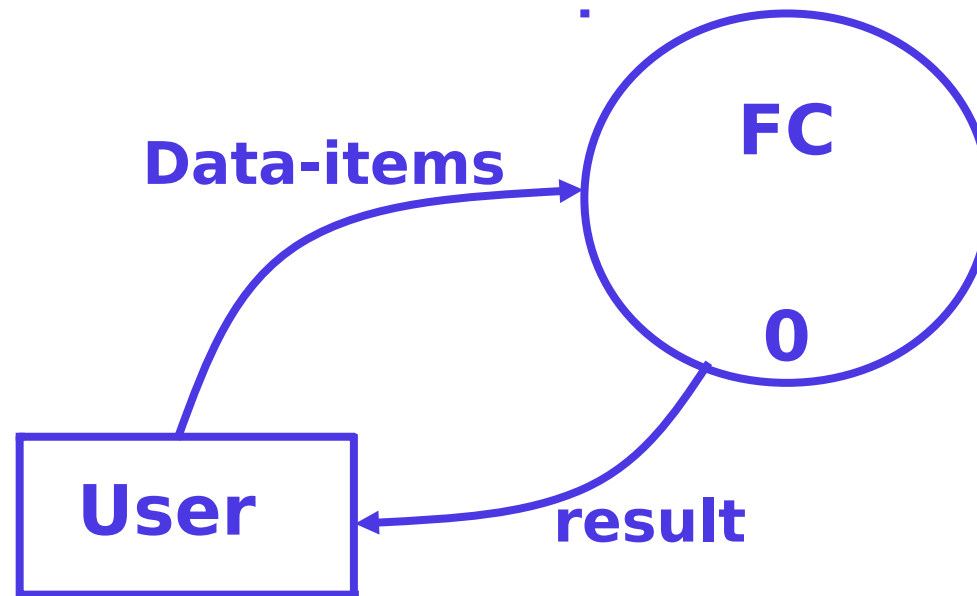
- Consider a software called formulae calculator software
 - Reads three integers in the range of -1000 and +1000
 - Finds out the root mean square of the three input numbers
 - Displays the result.
- The context diagram is simple to develop:
 - The system accepts 3 integers from the user
 - Returns the result to him.
- From a cursory analysis of the problem description:
 - We can see that the system needs to perform several things.
 - Accept input numbers from the user:
 - Validate the numbers,
 - Calculate the root mean square of the input numbers
 - Display the result.

Structured Analysis (SA)

Input: Requirements Doc / SRS

Output: Different DFDs

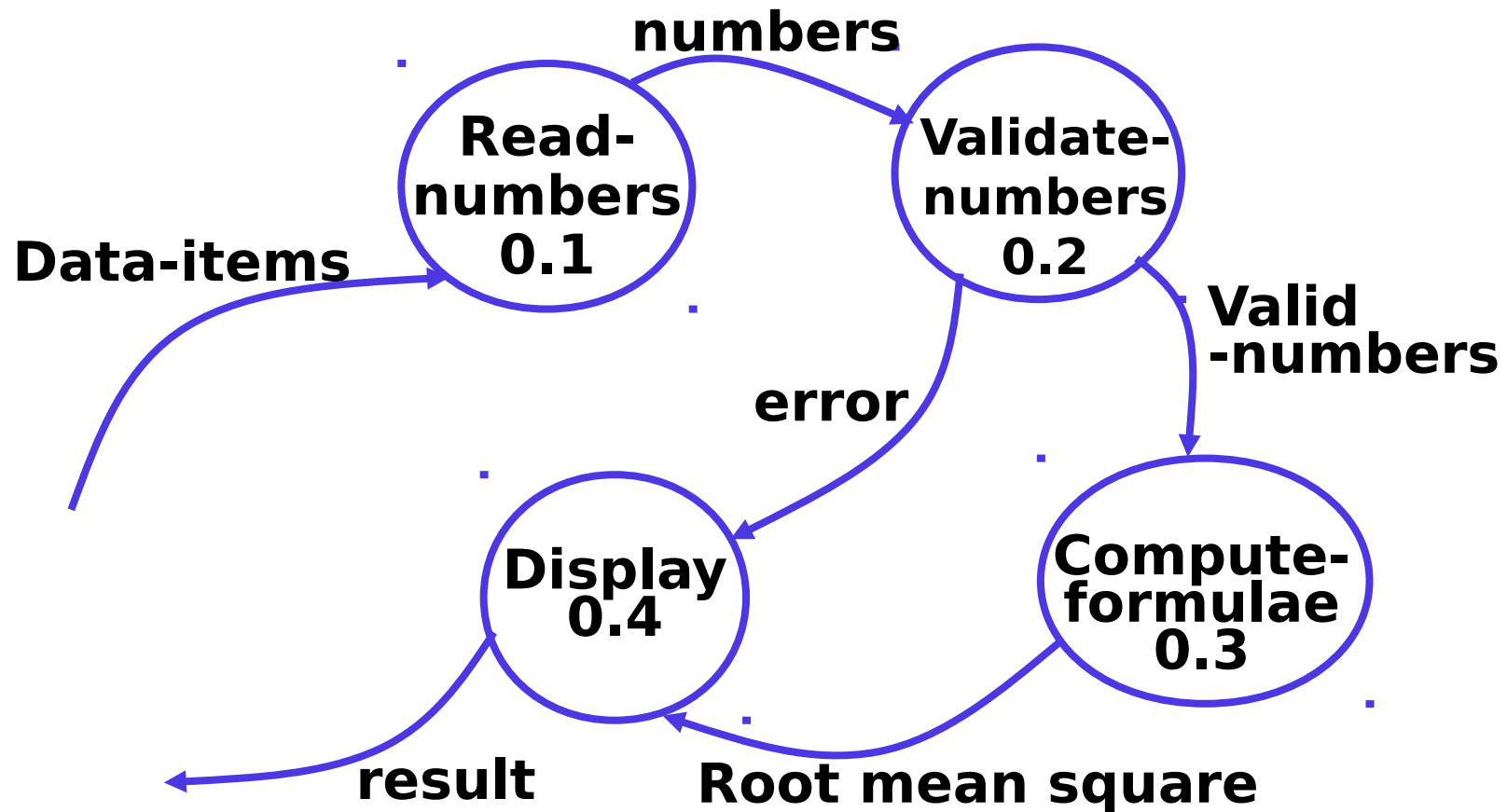
Formulae Calculator (FC) Software : Context Diagram



Level 0 DFD

Formulae Calculator (FC) Software :

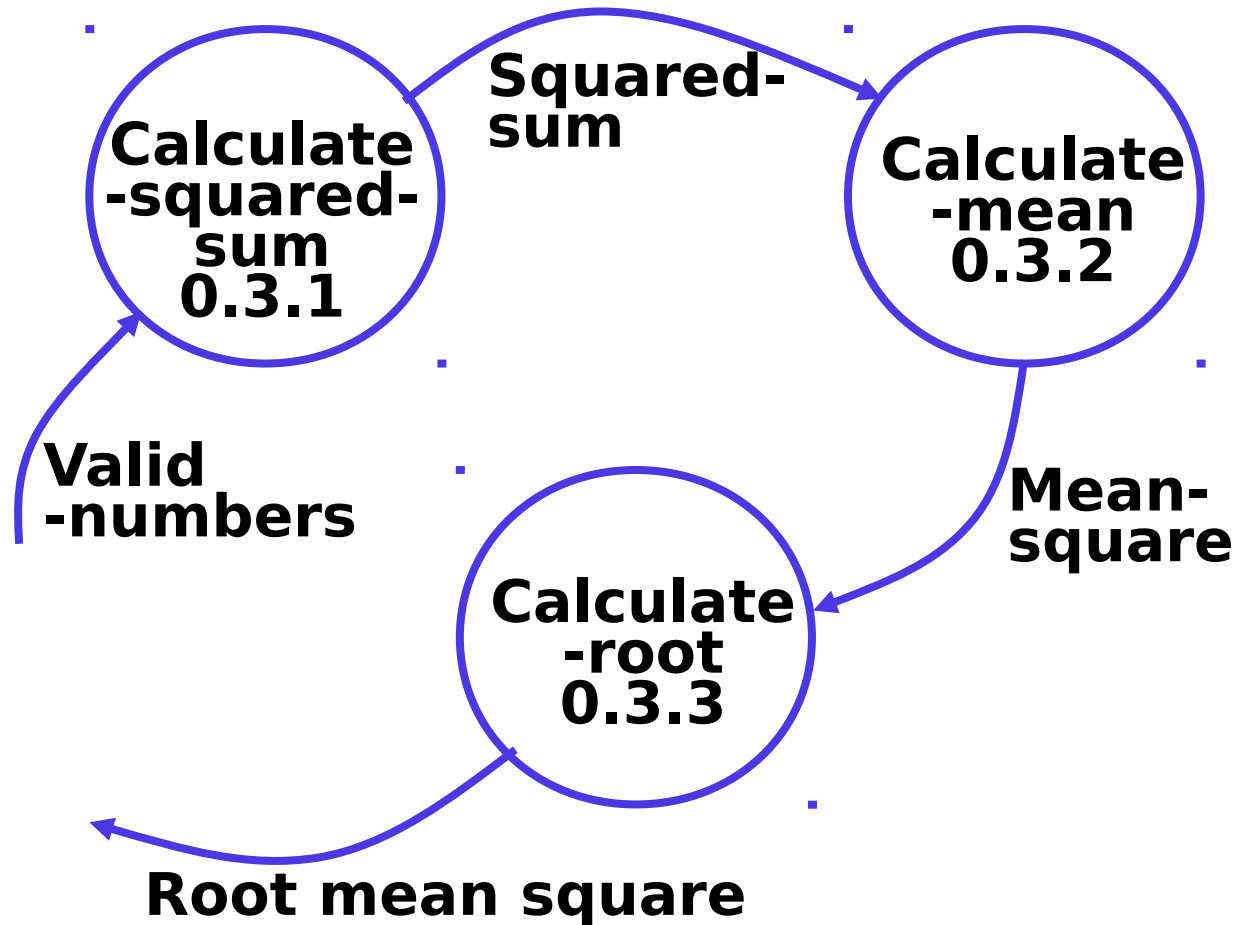
Level 1 DFD



Note: 0.1, 0.2, 0.4 are operations that no more require decomposition.
However, 0.3 can be further decomposed. Hence Level 2 DFD for 0.3 in next slide.

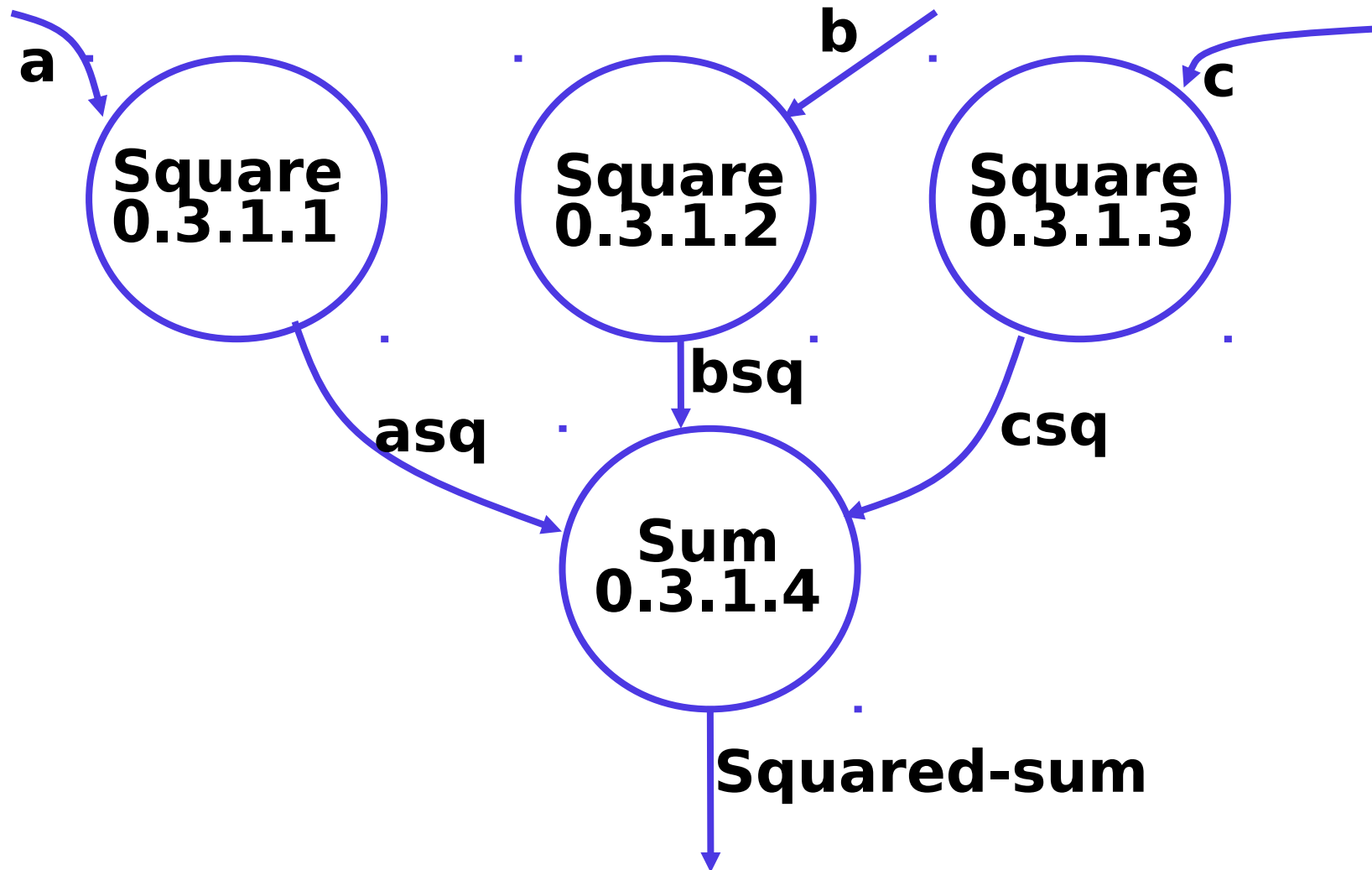
Formulae Calculator (FC) Software :

Level 2 DFD



Formulae Calculator (FC) Software :

Level 3 DFD (for 0.3.1 in level 2 DFD)



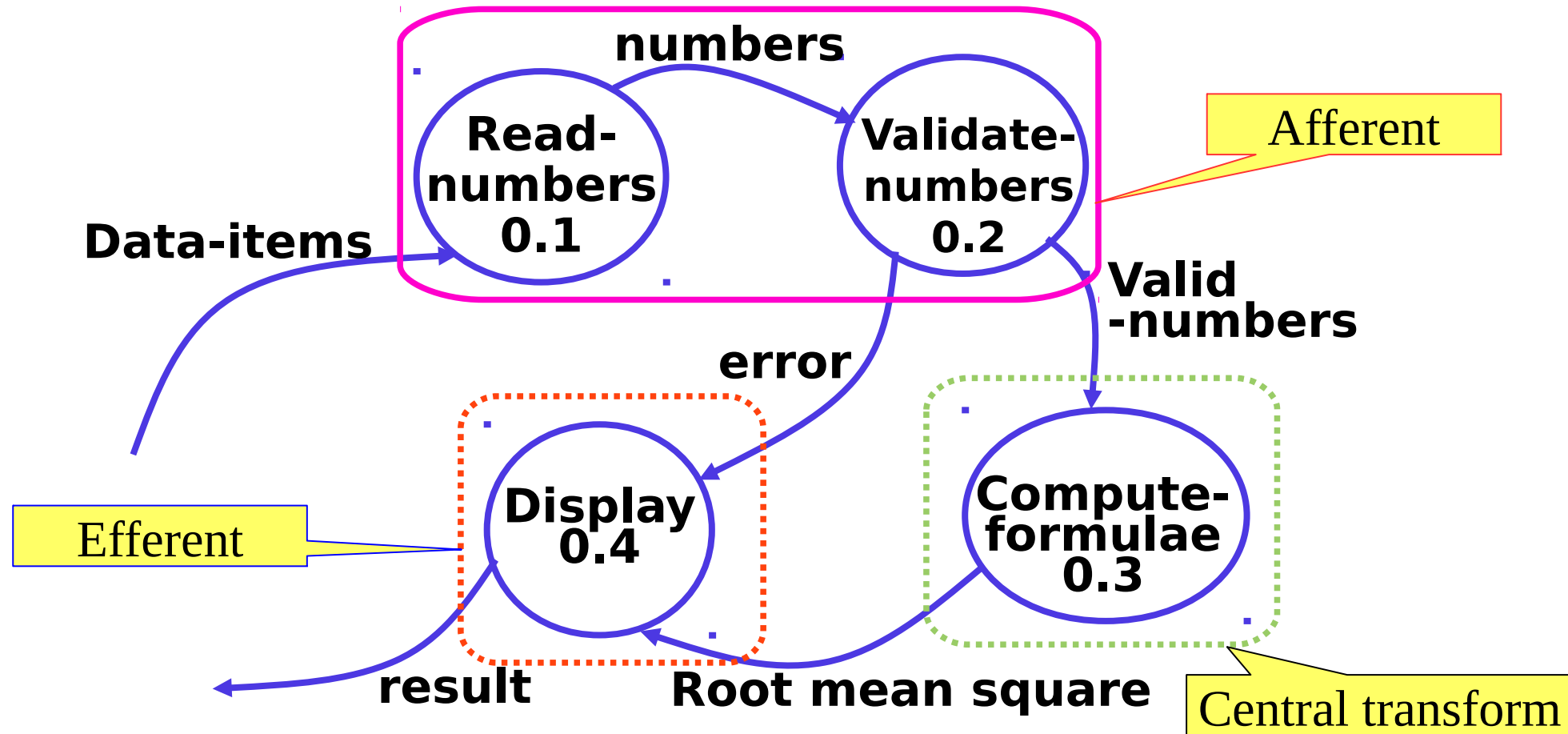
Note: You may add Level 3 DFD for 0.3.2 and 0.3.3, if needed.

Structured Design (SD)

Input: DFD obtained in SA phase

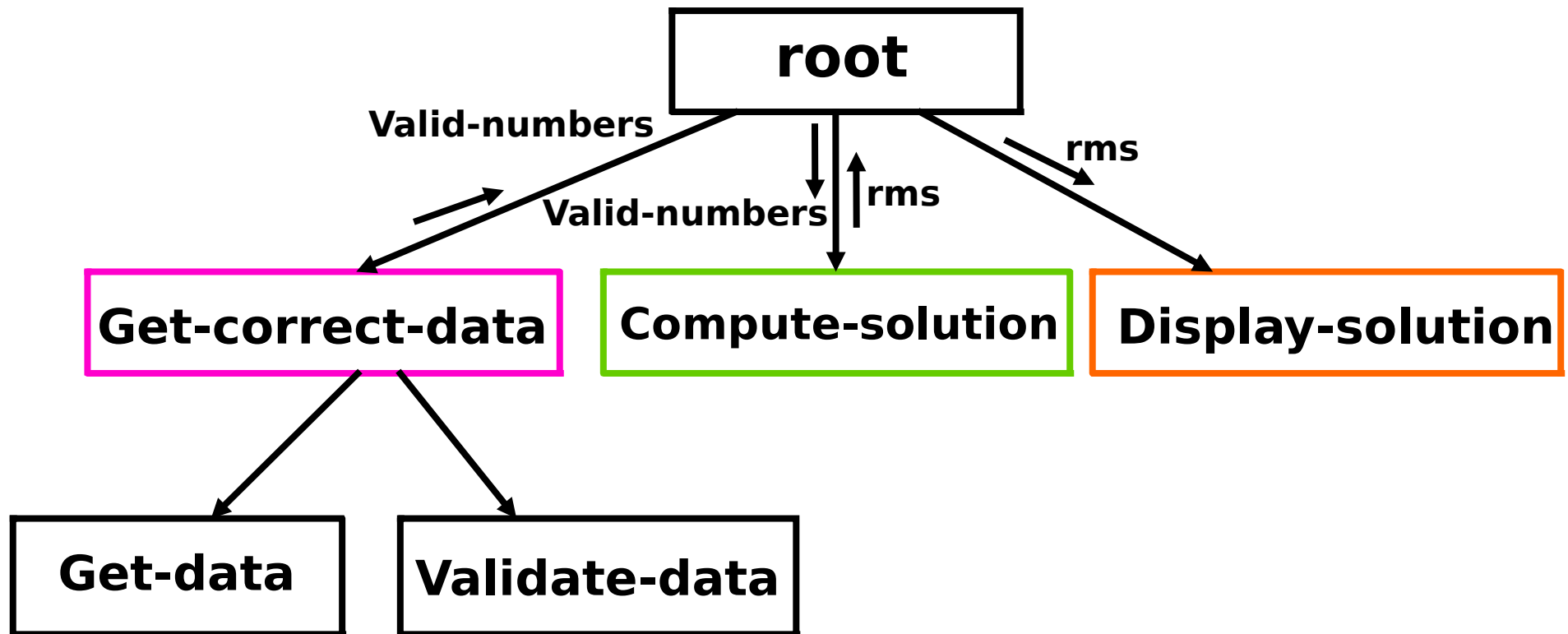
Output: Module Structure Chart and Data Dictionary

Formulae Calculator (FC) Software : Process of Structured Design



- By observing the level 1 DFD:
 - Identify 0.1 and 0.2 bubbles as the afferent branch – Input section
 - Display 0.4 as the efferent branch – output section
 - Compute-formulae 0.3 is the remaining portion as the central transform – logic processing

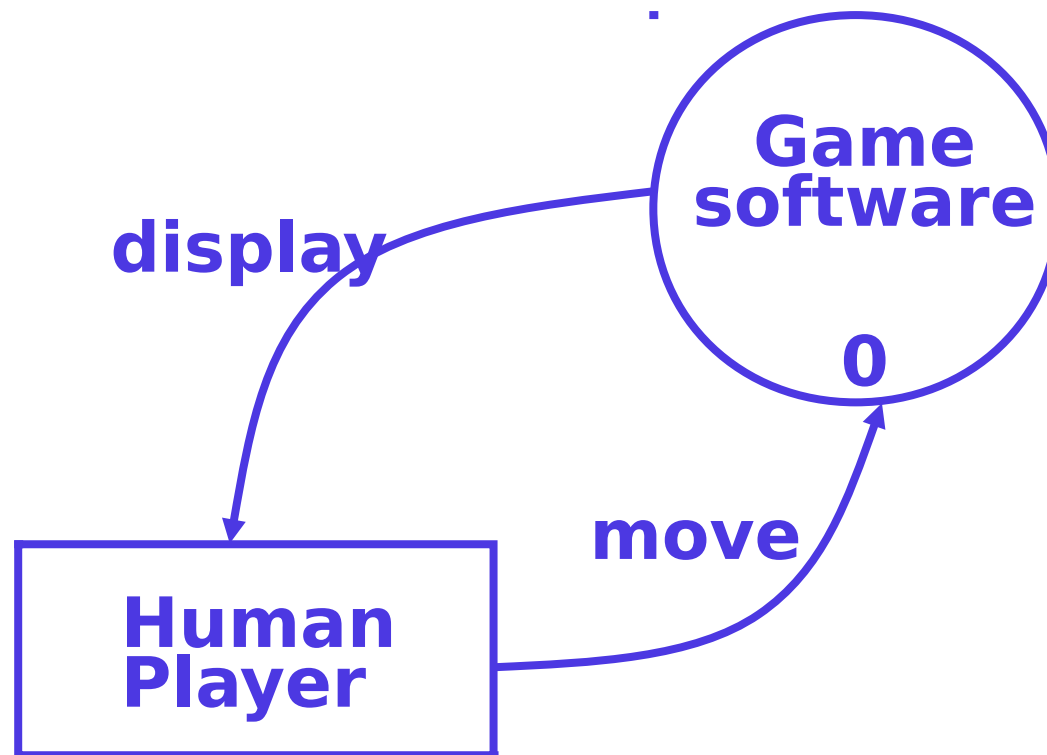
Formulae Calculator (FC) Software : Module Structure Chart



Example : Game Software

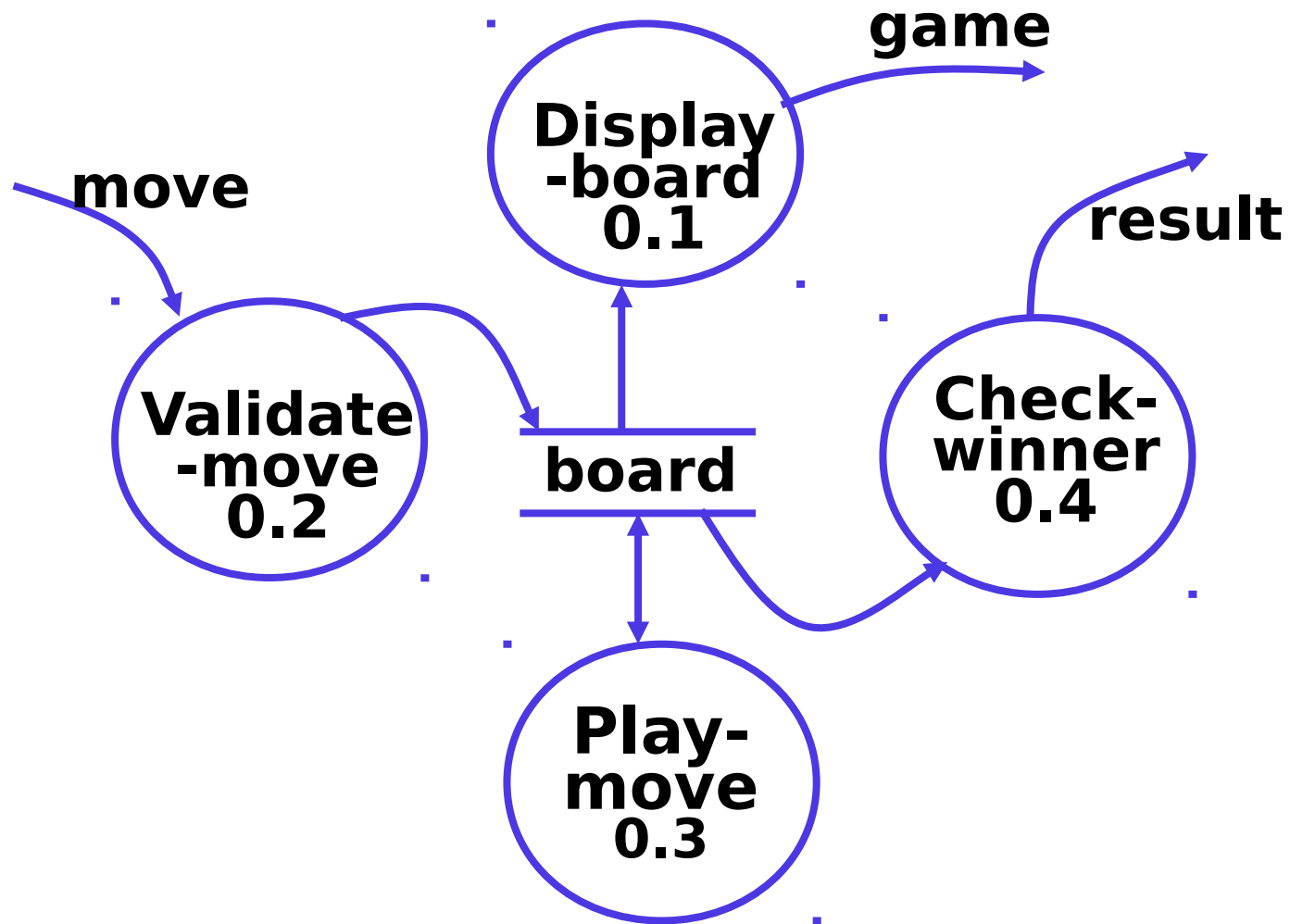
- A human player and the computer make alternate moves on a 3 X 3 square.
- A move consists of marking a previously unmarked square.
- The user inputs a number between 1 and 9 to mark a square.
- Whoever is first to place three consecutive marks along a straight line (i.e., along a row, column, or diagonal) on the square wins.
- As soon as either of the human player or the computer wins,
 - A message announcing the winner should be displayed.
- If neither player manages to get three consecutive marks along a straight line,
 - And all the squares on the board are filled up, then the game is drawn.
- The computer always tries to win a game.

Game Software : Context Diagram

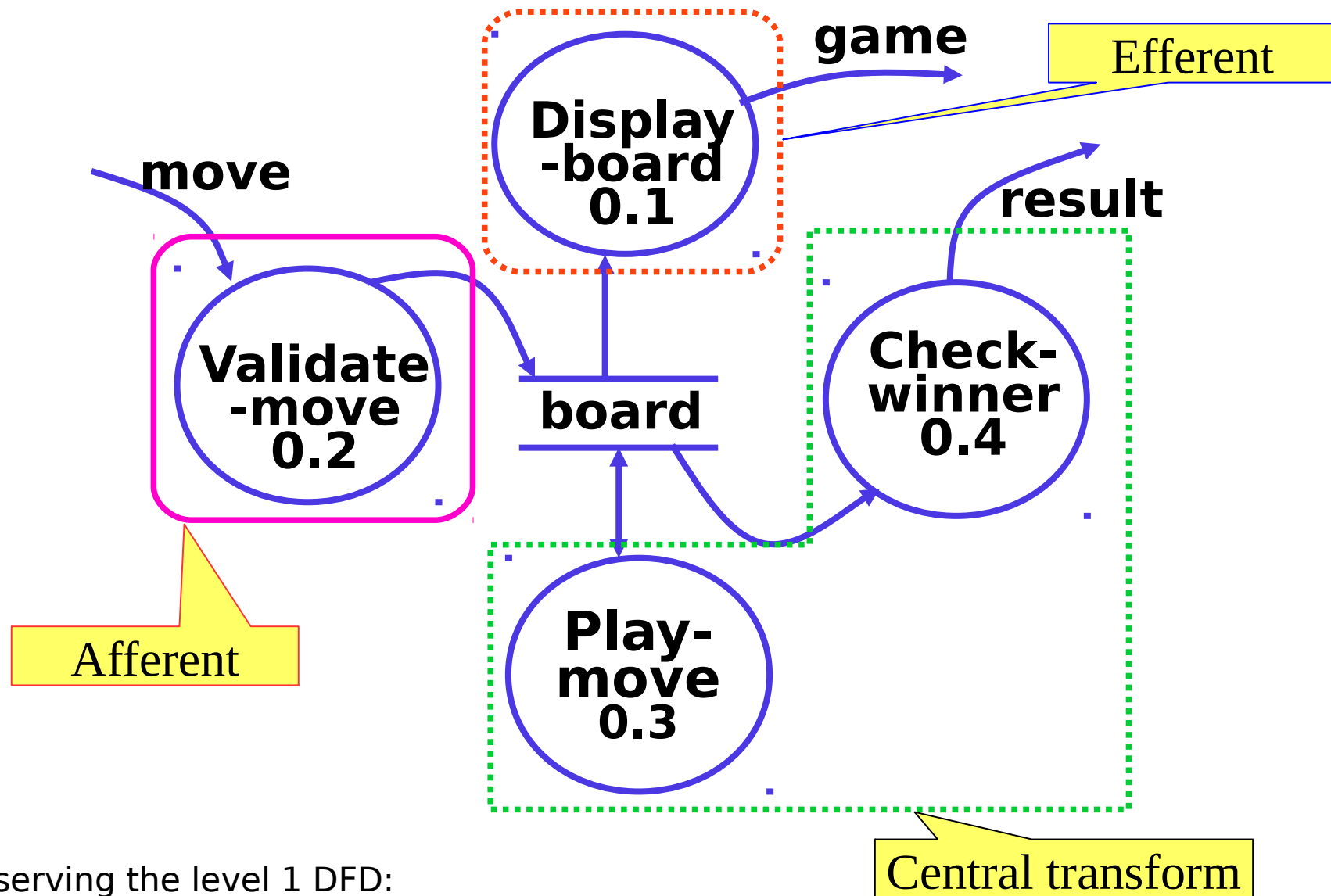


Level 0 DFD

Game Software : Level 1 DFD

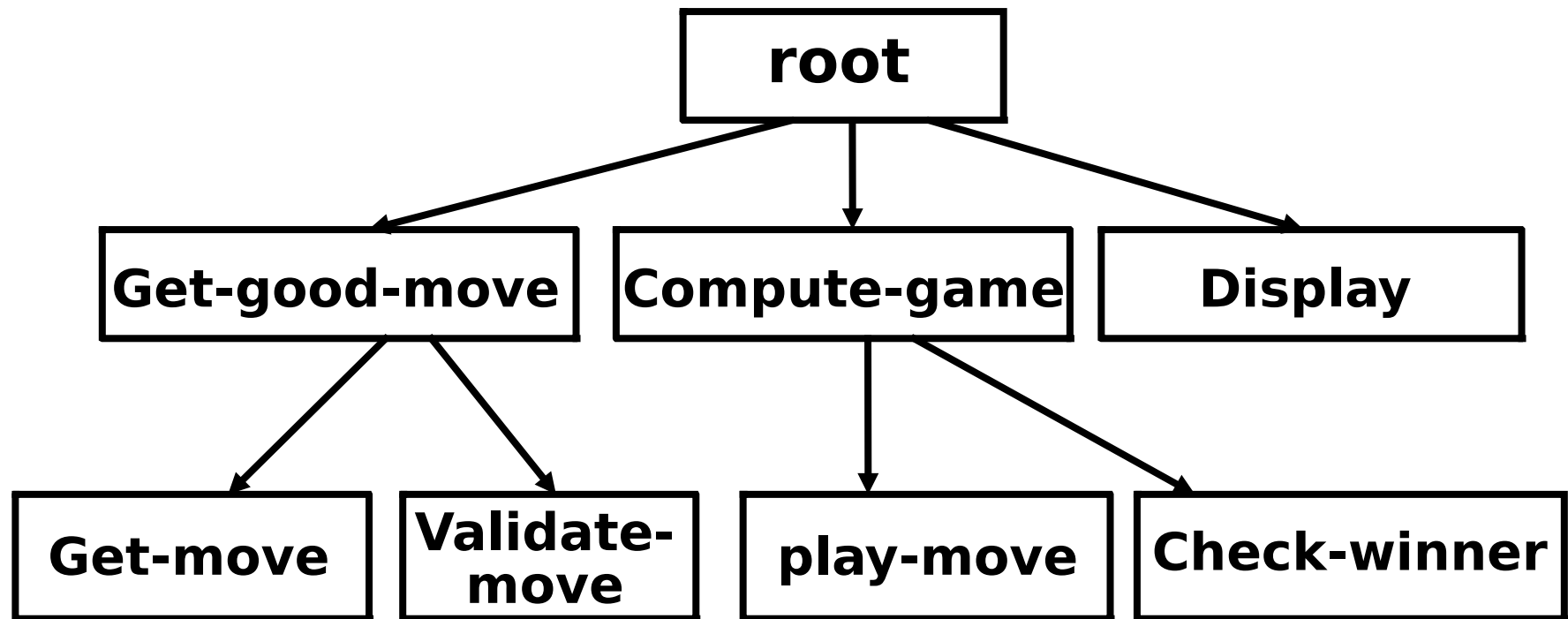


Using Level 1 DFD for SD to construct Module Structure Chart



- By observing the level 1 DFD:
 - Identify 0.2 bubble as the afferent branch - Input section
 - Display 0.1 as the efferent branch - output section
 - Play-move (0.3) and Check-winner (0.4) is the remaining portion as the central transform - logic processing

Game Software :Structure Chart



Game Software : Data Dictionary

- Display = game + result
- move = integer
- board = {integer}9
- game = {integer}9
- result = string

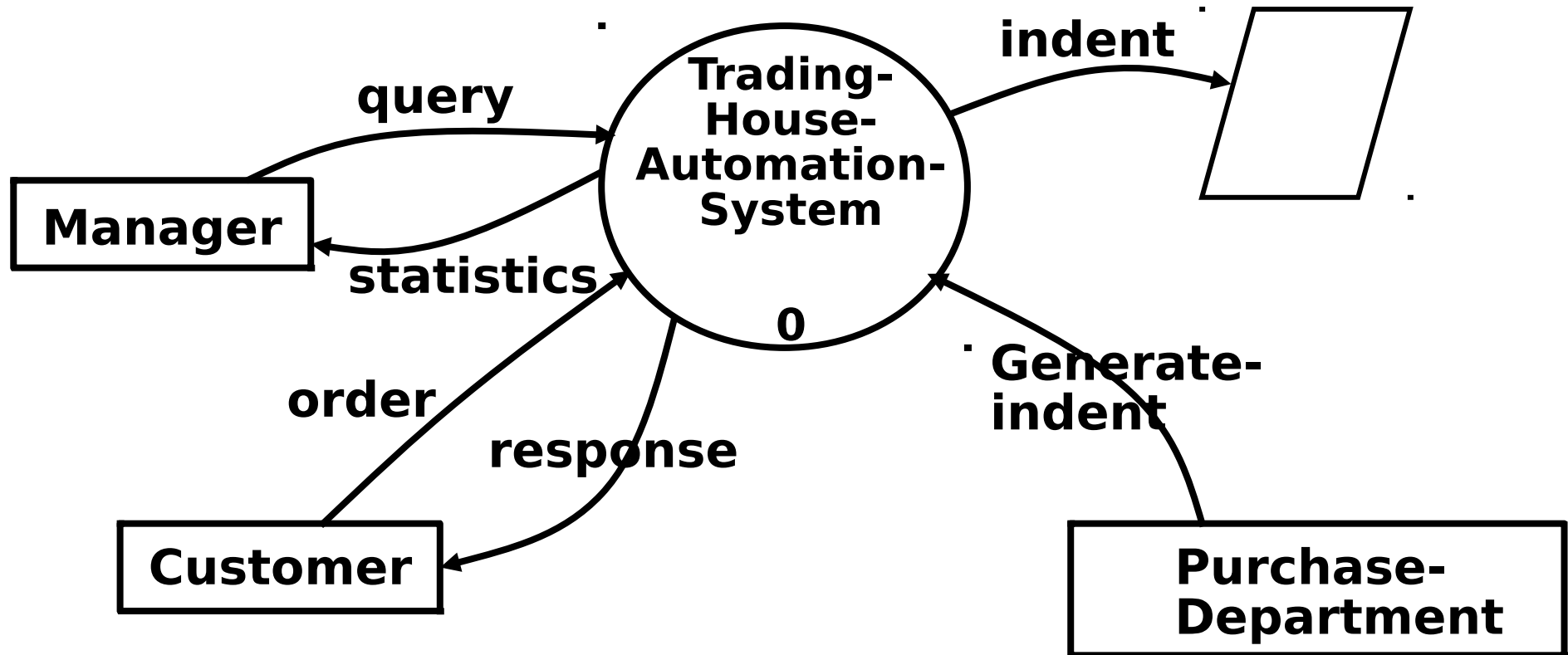
Example: Trading-House Automation System

- A large trading house wants us to develop a software to automate book keeping activities associated with its business. It has many regular customers who place orders for various kinds of commodities.
- The trading house maintains names and addresses of its regular customers. Each customer is assigned a unique customer identification number (CIN). As per current practice when a customer places order, the accounts department first checks the credit-worthiness of the customer.
- The credit worthiness of a customer is determined by analyzing the history of his payments to the bills sent to him in the past. If a customer is not credit-worthy, his orders are not processed any further. An appropriate order rejection message is generated for the customer.
- If a customer is credit-worthy, items he/she has ordered are checked against the list of items the trading house deals with. The items that the trading house does not deal with are not processed any further. An appropriate message for the customer for these items is generated.
- The items in a customer's order that the trading house deals with are checked for availability in inventory. If the items are available in the inventory in desired quantities, a bill with the forwarding address of the customer is printed. A material issue slip is printed.

Example: Trading-House Automation System

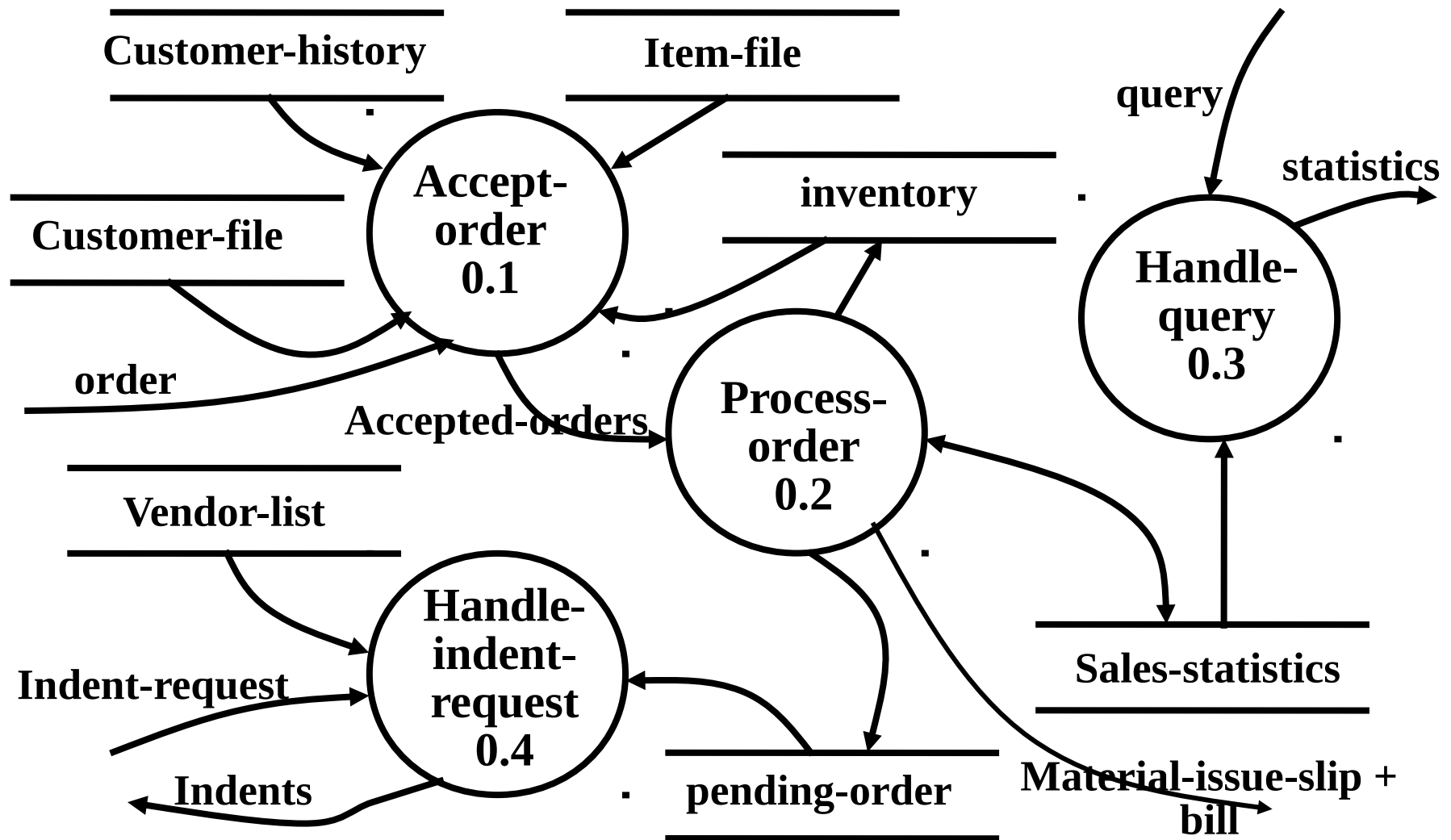
- The customer can produce the material issue slip at the store house, take delivery of the items and the inventory data is adjusted to reflect the sale to the customer.
- If an ordered item is not available in the inventory in sufficient quantity, and to be able to fulfill pending orders, the system stores details in a "pending-order" file including out-of-stock items along with quantity ordered and customer identification number.
- The purchase department would periodically issue commands to generate indents. When generate indents command is issued, the system should examine the "pending-order" file, determine the orders that are pending and the total quantity required for each of the items.
- TAS should find out the addresses of the vendors who supply the required items. Examine the file containing vendor details (their address, items they supply etc.) and print out indents to those vendors.
- TAS should also answer managerial queries such as statistics of different items sold over any given period of time, corresponding quantity sold and the price realized.

Context Diagram

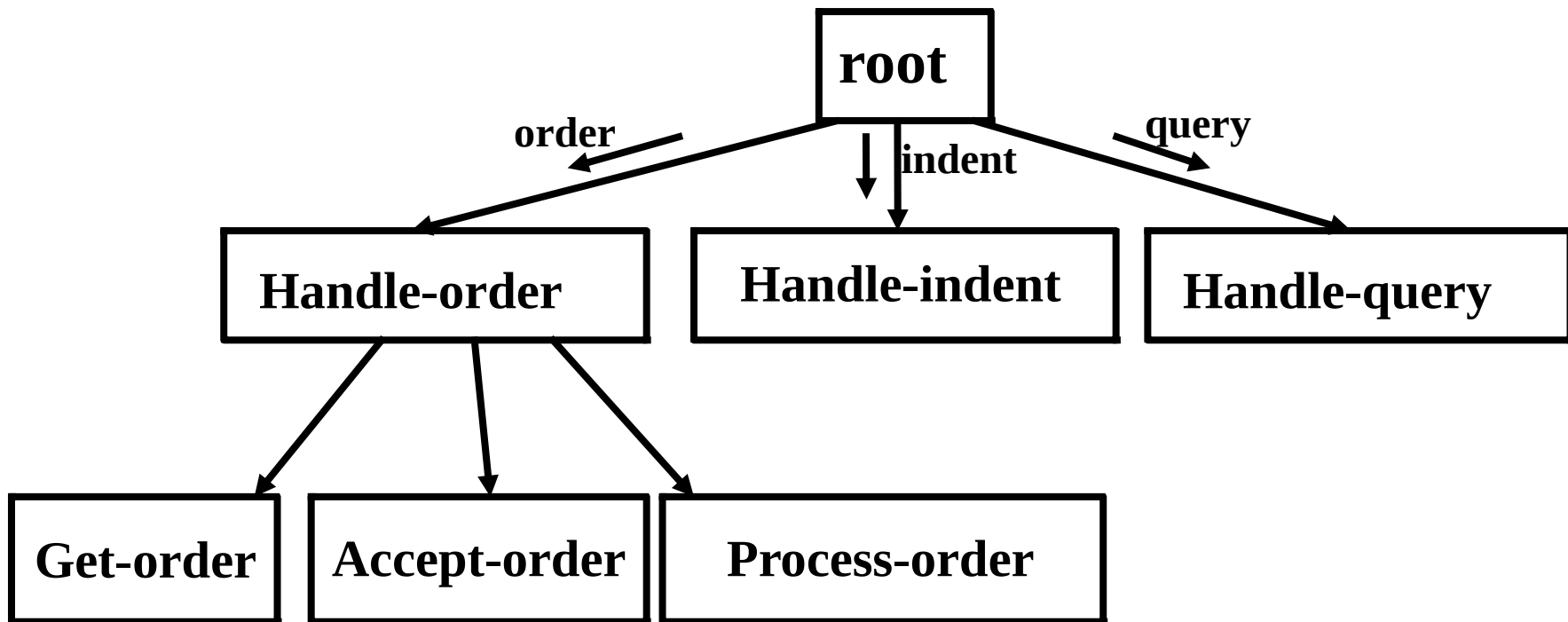


Level 0 DFD

Level 1 DFD



Structure Chart



Add more details (if any) to the above structure chart to give an elaborate view.

Example: Sample Data Dictionary

- response: [bill + material-issue-slip, reject-message]
- query: period /* query from manager regarding sales statistics*/
- period: [date+date,month,year,day]
- date: year + month + day
- year: integer
- month: integer
- day: integer
- order: customer-id + {items + quantity}*
- accepted-order: order /* ordered items available in inventory */
- reject-message: order + message /* rejection message */
- pending-orders: customer-id + {items+quantity}*
- customer-address: name+house#+street#+city+pin
- item-name: string
- house#: string
- street#: string
- city: string
- pin: integer
- customer-id: integer
- bill: {item + quantity + price}* + total-amount + customer-address
- material-issue-slip: message + item + quantity + customer-address
- message: string
- statistics: {item + quantity + price }*
- sales-statistics: {statistics}*
- quantity: integer